Dublin City University

School of Computing

MSc Computing (Artificial Intelligence)

Foundations of Artificial Intelligence

Empirical Evaluation of Hallucinations in Large Language Models

A Comparative Study of Code Generation and Technical Knowledge Tasks

---

Arun Narayanan

arun.narayanan2@mail.dcu.ie

November 2025

Word count: ∼5,500 (excluding tables and references)

Contents

Abstract **3**

1 Introduction **4**

2 Related Work **4**
2.1 Hallucination Definitions 4
2.2 Code Evaluation Benchmarks 5
2.3 Knowledge and Truthfulness Benchmarks 5
2.4 Domain-Specific LLM Evaluation 5
2.5 How This Assignment Relates to Existing Work 5

3 Methodology **6**
3.1 Testing Framework Design 6
3.2 Dataset Composition 6
3.3 AI Models Tested 7
3.4 Evaluation Criteria 7
3.5 Prompt Engineering 8
3.6 Statistical Methods 8

4 Results **8**
4.1 Overall Performance 8
4.2 Performance by Mode 8
4.3 Performance by Category 9
4.4 Format Violations and Refusals 10
4.5 Failure Mode Analysis 10

5 Discussion **11**
5.1 Why Knowledge Tasks Are Harder 11
5.2 Specialised Domains: Snowflake at 40% 11
5.3 Hallucination as the Dominant Failure Mode 11

6 Limitations **12**
6.1 Construct Validity 12
6.2 Internal Validity 12
6.3 External Validity 12
6.4 Conclusion Validity 13
6.5 Dataset Authorship 13
6.6 Training Data Cutoffs 13

7 Future Work **13**

8 Conclusion **13**

References **15**

List of Tables

1 Overall Performance Summary 8
2 Performance by Evaluation Mode 9
3 Python Code Category Performance 9
4 Knowledge Domain Category Performance 9

Abstract

Large Language Models (LLMs) are increasingly deployed in software development and technical workflows, yet their tendency to generate plausible but incorrect outputs—known as hallucinations—remains a critical concern. This report presents an empirical evaluation of hallucination rates in two LLMs: Google Gemini 2.5 Flash and Cohere Command R7B. A dual-mode testing framework was developed combining objective code execution testing (50 questions) with knowledge-based questions (50 questions) across eight technical domains. The evaluation distinguishes between *hallucinations* (factually incorrect or logically flawed answers), *format violations* (correct answers embedded in extra text), and *refusals* (model declines to answer or returns an empty response). Gemini achieved 89% overall accuracy while Cohere achieved 64%, a 25 percentage-point gap. Code execution tasks proved substantially easier (90% average) than specialised technical knowledge tasks (63% average). Performance degraded markedly for specialised domains, particularly Snowflake (40% average) and Spark (55% average). These findings reveal notable gaps in specialised technical knowledge with practical implications for software development and data engineering workflows.

1 Introduction

Large Language Models, including GPT-4 [1] and Cohere Command R7B [3], are now routinely employed for tasks ranging from general writing to specialised domains including software development, data engineering, and database management. A well-documented limitation of LLMs is their propensity to generate hallucinations: plausible-sounding outputs that are factually incorrect or logically flawed [5]. This presents serious risks in technical domains where incorrect code or misinformation can lead to security vulnerabilities, system failures, and data loss.

Many existing evaluations of LLM performance rely on subjective human judgement or multiple-choice assessments [4, 6], both of which suffer from limitations. In contrast, code generation and technical knowledge retrieval offer opportunities for objective, reproducible evaluation. When an LLM generates Python code, that code can be executed in a controlled environment and compared against expected outputs with binary precision.

This assignment investigates three specific questions:

1. How do state-of-the-art LLMs perform on objective coding tasks with clear ground truth?
2. Do hallucination rates differ between code generation and factual knowledge retrieval tasks?
3. Which technical domains are most prone to hallucinations, and what patterns emerge?

To address these questions, a comprehensive testing framework was developed incorporating 100 questions spanning eight technical categories. The framework uses two evaluation modes: Python code execution via the Piston API sandbox for objective output verification, and direct string comparison for knowledge-based questions.

An additional aspect of this work is the explicit distinction between **hallucinations** (factually incorrect or logically flawed outputs), **format violations** (correct answers embedded in unnecessary explanations or preambles), and **refusals** (cases where the model declines to answer or returns an empty response). Some prior work conflates these categories [5], leading to imprecise characterisation of LLM failure modes.

2 Related Work

2.1 Hallucination Definitions

Ji et al. [5] provide a comprehensive survey distinguishing intrinsic hallucinations (content that contradicts the source input) from extrinsic hallucinations (content that cannot be

verified from the source). This evaluation focuses primarily on extrinsic hallucinations, where models generate content unsupported by the relevant source knowledge or ground truth. Maynez et al. [8] study faithfulness in abstractive summarisation, demonstrating that factual correctness and surface-level output quality are distinct dimensions of generation quality. Inspired by this distinction, the present evaluation separately tracks format violations, hallucinations, and refusals.

2.2 Code Evaluation Benchmarks

Chen et al. [2] present HumanEval, establishing execution-based code evaluation as a standard methodology. Liu et al. [7] extend this with HumanEval+, adding robustness through diverse test cases. The Python evaluation in this assignment follows the HumanEval paradigm but focuses on edge cases and evaluates commercial LLMs rather than open-source models.

2.3 Knowledge and Truthfulness Benchmarks

Hendrycks et al. [4] introduce MMLU, testing knowledge across 57 domains via multiple-choice questions. Lin et al. [6] present TruthfulQA, evaluating factual alignment with open-ended answers. Min et al. [9] introduce FactScore for fine-grained evaluation of factual precision. This assignment complements these benchmarks by evaluating specialised technical domains (Polars, Snowflake, Spark) that are underrepresented in existing benchmarks.

2.4 Domain-Specific LLM Evaluation

Rozière et al. [10] present Code Llama, evaluated on programming benchmarks. Gomez [3] introduces the Cohere Command R7B model. The present work uniquely evaluates *modern data engineering domains* (Snowflake, Polars, Spark) that are underrepresented in prior benchmarks.

2.5 How This Assignment Relates to Existing Work

This evaluation builds on existing work in four ways: (1) a four-category response taxonomy (correct, format violation, hallucination, refused) providing granular failure analysis; (2) specialised technical domain evaluation extending beyond foundational domains; (3) comparative binary evaluation demonstrating that code tasks are substantially easier than knowledge retrieval; and (4) clear, reproducible methodology compatible with the HumanEval [2] and TruthfulQA [6] paradigms.

3 Methodology

3.1 Testing Framework Design

Mode 1: Python Code Execution (Objective Scoring). LLMs are prompted to generate complete Python code solving specific algorithmic and logical challenges. The generated code is transmitted to the Piston API (a cloud-based code execution sandbox) where it is compiled and executed. A 5-second timeout was configured for each execution to prevent infinite loops. A response is marked correct only if: (a) the code compiles without syntax errors, (b) the code executes without runtime errors, and (c) the output exactly matches the expected result character-for-character.

Mode 2: Knowledge-Based Questions (String Matching with Format Detection). LLMs are prompted to answer technical questions with brief, precise answers. The answer is compared against ground truth using: (1) exact matching, (2) partial matching fallback, and (3) format violation detection. Four response categories are distinguished:

- **Correct**: Answer exactly matches expected output.
- **Format Violation**: Correct answer present but with extra text (e.g., "The answer is DISTINCT" instead of "DISTINCT").
- **Hallucination**: Factually incorrect or logically flawed answer.
- **Refused**: Model declines to answer or returns an empty response.

System prompts explicitly instruct models to provide only answers without explanations, allowing format compliance violations to be quantified separately.

3.2 Dataset Composition

The dataset consists of **100 questions** divided equally between two modes.

Python Code Execution Questions (50 total):

- **Edge Cases** (15): floating-point arithmetic, type comparison, identity vs. equality, truthiness evaluation.
- **String Tricks** (10): reversal, character counting, whitespace handling, palindrome detection.
- **Algorithms** (15): prime detection, GCD/LCM, factorial computation, perfect squares, divisor sums.
- **Data Structures** (10): list slicing, dictionary operations, set operations, matrix transposition.

Approximately 60% are classified as *Hard* difficulty, targeting edge cases to maximise discrimination between models.

Knowledge-Based Questions (50 total):

- **Polars** (15): modern DataFrame library covering lazy evaluation (`scan_csv`, `collect`), column operations, conditional operations, joins, and window functions.
- **SQL** (15): standard SQL including `SELECT`, `JOIN`, `GROUP BY`, aggregate functions, and set operations.
- **Spark** (10): distributed computing concepts including actions, transformations, persistence levels, and shuffle operations.
- **Snowflake** (10): cloud warehouse features including `QUALIFY`, `MAX_BY`, `GROUP BY ALL`, `EXCLUDE`, Time Travel, and `FLATTEN`.

The dataset intentionally includes specialised, modern tools (Polars, Snowflake) that may be less familiar to current models, enabling evaluation of model behaviour at the frontier of their knowledge.

3.3 AI Models Tested

Google Gemini 2.5 Flash: Released June 2025, a lightweight multimodal transformer with a 1-million-token context window, accessed via the Google Gemini API.

Cohere Command R7B: Released December 2024, a 7-billion-parameter decoder-only model designed for enterprise deployments [3], accessed via the Cohere v2 Chat API (model identifier: `command-r7b-12-2024`).

Both models were evaluated through official commercial APIs with identical prompts and evaluation criteria. No fine-tuning or model-specific optimisations were applied.

3.4 Evaluation Criteria

Python Code Responses: Syntax correctness, runtime correctness, output correctness, and timeout handling (5-second limit). A response is correct if and only if all criteria pass.

Knowledge Question Responses: Exact match → Correct; partial match without explanation → Correct; correct answer with appended explanation → Format Violation; incorrect answer → Hallucination; no answer or refusal → Refused. Format violations, hallucinations, and refusals are tracked separately.

3.5 Prompt Engineering

Python mode: *"Write complete Python code that solves this problem. Only provide the code, no explanations."*

Knowledge mode: *"Answer with ONLY the exact function name, keyword, or term. No explanations, no extra text."*

Both prompts were deliberately straightforward, designed to test models on their default behaviour rather than highly engineered responses.

3.6 Statistical Methods

All reported accuracy percentages include 95% confidence intervals calculated using the exact binomial distribution. Category-level differences are tested using Fisher's exact test (for groups with $n < 20$). Due to the small per-category sample sizes ($n = 10$–15), individual category comparisons have limited statistical power; observed differences are reported descriptively.

4 Results

4.1 Overall Performance

Gemini 2.5 Flash achieved **89% overall accuracy** (89/100), while Cohere Command R7B achieved **64% overall accuracy** (64/100), a **25 percentage-point gap**. The 95% confidence intervals do not overlap (Gemini: [81–94%]; Cohere: [54–73%]), strongly suggesting that the overall performance difference is statistically meaningful.

Table 1: Overall Performance Summary

Metric	**Gemini**	**Cohere**	**Δ**
Overall Accuracy	89%	64%	+25 pp
95% CI	[81–94%]	[54–73%]	–
Correct Responses	89	64	+25
Format Violations	1	6	−5
Hallucinations	9	28	−19
Refused	1	2	−1

All four response categories are tracked separately. Cohere's hallucination rate (28%) is substantially higher than Gemini's (9%).

4.2 Performance by Mode

Performance diverges sharply between code execution and knowledge tasks (Table 2).

Table 2: Performance by Evaluation Mode

Mode	**Gemini**	**Cohere**	**Avg**
Python Code	96% (48/50)	84% (42/50)	90%
Knowledge	82% (41/50)	44% (22/50)	63%
Overall	**89%**	**64%**	**77%**

The 27 pp gap between Python (90%) and Knowledge (63%) average accuracy indicates that objective, syntax-constrained tasks are substantially easier for LLMs.

The 27 percentage-point difference between Python (90% average) and Knowledge (63% average) demonstrates that objective, rule-based tasks with clear syntax constraints are substantially easier for LLMs than open-ended knowledge retrieval.

4.3 Performance by Category

Tables 3 and 4 present category-level results with observed performance differences.

Table 3: Python Code Category Performance

Category	**Gem.**	**Coh.**	**Avg**	**Δ**
Edge Case (15)	93%	80%	87%	+13 pp
String Trick (10)	100%	90%	95%	+10 pp
Algorithm (15)	100%	87%	93%	+13 pp
Data Structure (10)	90%	80%	85%	+10 pp
Average	**96%**	**84%**	**90%**	**+12 pp**

None of the Python category differences are statistically significant via Fisher's exact test, reflecting relatively similar performance on code tasks.

Table 4: Knowledge Domain Category Performance

Category	**Gem.**	**Coh.**	**Avg**	**Δ**
Polars (15)	87%	53%	70%	+34 pp
SQL (15)	87%	80%	83%	+7 pp
Spark (10)	80%	30%	55%	+50 pp
Snowflake (10)	70%	10%	40%	+60 pp
Average	**82%**	**44%**	**63%**	**+38 pp**

Per-category sample sizes ($n = 10$–15) are too small for individual categories to reach statistical significance via Fisher's exact test after correcting for multiple comparisons. Differences are reported descriptively.

The large observed gaps in specialised domains (Snowflake: 60 pp, Spark: 50 pp) are noteworthy, though per-category sample sizes limit formal statistical inference. Differences in fundamental domains (SQL: 7 pp) are small.

4.4 Format Violations and Refusals

A small but notable proportion of responses were classified as *format violations* (correct answer with extra text) rather than hallucinations. Gemini produced 1 format violation (1%) while Cohere produced 6 (6%). Additionally, Gemini refused 1 question (1%) and Cohere refused 2 (2%). Two illustrative format violation examples:

Example 1 (Knowledge). Question: "What SQL keyword retrieves unique values?" Expected: `DISTINCT`. Cohere responded: "SELECT DISTINCT to retrieve unique values from a column." Classification: Format Violation (correct answer embedded in explanation).

Example 2 (Knowledge). Question: "What Polars method scans a CSV file lazily?" Expected: `scan_csv`. Cohere responded: "Polars offers the scan_csv method for lazy loading of CSV files." Classification: Format Violation (correct answer present but with extra context).

Although format violations represent a small fraction of total errors (1% for Gemini, 6% for Cohere), the distinction matters: format violations indicate that knowledge is present but instruction compliance is lacking, whereas hallucinations reflect genuinely incorrect information.

4.5 Failure Mode Analysis

Gemini's failures (11 total: 1 format violation, 9 hallucinations, 1 refused): Hallucinations include incorrect identity comparisons, wrong modulo results, and confused function names (`filter` vs. `select` in Polars). The single format violation involved a knowledge answer wrapped in a brief explanation. The refused response was a question where the model indicated insufficient context.

Cohere's failures (36 total: 6 format violations, 28 hallucinations, 2 refused): Hallucinations include plausible but incorrect code, confused function names in Spark and Snowflake, and incorrect terminology. Format violations were concentrated in knowledge questions where correct answers were wrapped in explanations. The 28 hallucinations represent the primary failure mode, with Snowflake (9 errors) and Spark (7 errors) contributing the most.

5 Discussion

5.1 Why Knowledge Tasks Are Harder

Performance on knowledge tasks (63% average) lags code execution (90% average) by 27 percentage points. Four factors contribute to this gap:

Domain Specialisation and Training Data. Polars (first released in 2020) and certain Snowflake-specific features such as `MAX_BY` and `GROUP BY ALL` are relatively recent additions that may post-date some LLM training cutoffs. Knowledge questions were intentionally selected from modern, specialised domains where models are expected to have gaps.

Terminology Precision. Knowledge answers require exact function or keyword names. LLMs frequently hallucinate plausible alternatives, such as `read_csv` instead of `scan_csv` (Polars), or `WHERE` instead of `QUALIFY` (Snowflake). Each alternative is syntactically plausible but incorrect.

Absence of Syntax Validation. Code execution has a built-in validation mechanism: syntax and runtime errors are caught immediately. Knowledge questions have no such compilation phase, so incorrect terminology passes silently.

Disambiguation Requirements. Knowledge questions reference functions within large multi-function libraries (Polars, for instance, has an extensive API with hundreds of functions). The LLM must disambiguate among semantically similar options, whereas code questions typically have a unique correct solution.

5.2 Specialised Domains: Snowflake at 40%

Snowflake achieved the worst category performance (Gemini 70%, Cohere 10%, average 40%). Likely contributing factors include: (1) enterprise tool specificity, as Snowflake is proprietary with documentation that may be less familiar to current models; (2) some features tested (e.g., `MAX_BY`, `GROUP BY ALL`) are relatively recent additions, potentially post-dating training cutoffs, while others such as `QUALIFY` have been available since at least 2021; (3) unique syntax patterns diverging from standard SQL; and (4) Cohere's smaller model size (7B parameters), which may contribute to reduced recall on knowledge-intensive tasks, though this evaluation cannot isolate parameter count as a causal variable.

5.3 Hallucination as the Dominant Failure Mode

The data reveals that hallucinations, not format violations, are the dominant failure mode for both models. Gemini produced 9 hallucinations (9%) compared to only 1 format violation (1%). Cohere produced 28 hallucinations (28%) compared to 6 format violations

(6%). This indicates that when models fail, they overwhelmingly produce factually incorrect answers rather than correct answers in non-compliant formatting.

Cohere's hallucination rate is more than three times Gemini's (28% vs. 9%), with the disparity concentrated in knowledge tasks. In the Snowflake category, Cohere hallucinated on 9 out of 10 questions, suggesting near-complete absence of domain knowledge. Gemini's hallucinations were spread more evenly across categories, suggesting broader but shallower knowledge gaps.

The small number of format violations (7 total across both models) suggests that, at least in this evaluation, modern LLMs generally comply with instruction formatting when they possess the relevant knowledge. This pattern is consistent with the hypothesis that the primary bottleneck is knowledge availability rather than instruction following, though the small sample of format violations limits the strength of this conclusion.

6 Limitations

6.1 Construct Validity

The 100 test questions focus on structured, objectively-scored tasks. Open-ended generation is not evaluated. Python questions emphasise syntax and logic, not algorithmic efficiency. Knowledge questions test terminology recall, not conceptual understanding. Results represent hallucination rates for *this specific evaluation domain*, not all possible LLM applications.

6.2 Internal Validity

A single prompt formulation per category is used; response patterns might change with different prompts. All generations used default temperature settings. Only zero-shot performance is evaluated; few-shot examples might improve results. The configured 5-second execution timeout for the Piston API sandbox may not reflect all production scenarios.

6.3 External Validity

Only two models were evaluated. Results do not generalise to other frontier LLMs. Model size disparity (Cohere 7B vs. Gemini's unknown but likely larger architecture) affects comparisons. The knowledge questions focus on modern data tools; generalisation to medical, legal, or scientific domains is unclear. All questions are English-only, and it is possible that hallucination rates would differ in multilingual settings where training data distribution varies further.

6.4 Conclusion Validity

Binary scoring misses partial correctness; future work could explore graded or partial-credit scoring schemes. Category sample sizes are small (n = 10–15), yielding wide confidence intervals. No inter-rater reliability study was conducted for knowledge questions; the exact-match criterion used for automated scoring is stricter than typical human judgement. For Python code execution, evaluation is fully deterministic and inter-rater reliability is not applicable.

6.5 Dataset Authorship

Test questions were created by the author without independent expert verification. Domain expert review would strengthen validity.

6.6 Training Data Cutoffs

Exact training data cutoffs for the models are not fully publicly documented (Cohere has disclosed partial cutoff information, while Google has not published Gemini's training cutoff), limiting the ability to distinguish training data gaps from architectural limitations.

7 Future Work

Key directions for extending this evaluation include:

1. Expanded model evaluation across a broader set of frontier models, including newer commercial and open-source LLMs.
2. Few-shot learning experiments with 1, 3, and 5 in-context examples.
3. Systematic prompt variation studies to assess sensitivity.
4. Temperature and sampling parameter sensitivity analysis.
5. Human evaluation with inter-rater reliability for knowledge questions.
6. Larger context evaluation to assess context-dependence of hallucination rates.

8 Conclusion

This empirical evaluation reveals clear hierarchies in LLM performance across technical domains.

Key Findings:

1. Gemini 2.5 Flash substantially outperforms Cohere Command R7B (89% vs. 64%), with the largest gaps in specialised domains (Snowflake: 60 pp gap).
2. Code execution tasks are substantially easier than knowledge retrieval (90% vs. 63% average), suggesting that objective constraints and syntax validation strongly improve LLM performance.
3. Hallucinations are the dominant failure mode (9% for Gemini, 28% for Cohere), while format violations are comparatively rare (1% and 6% respectively).
4. Specialised tools (Snowflake, Polars) expose substantial performance gaps, with accuracy degrading to 40% in the worst case.

Practical Implications: For critical applications (production code, database queries), human review remains essential, as neither model achieves near-perfect accuracy across all domains. In this evaluation, the dominance of hallucinations over format violations indicates that failures were primarily knowledge-related rather than formatting-related. Practitioners should not assume LLMs have reliable knowledge of modern specialised tools without domain-specific validation.

Summary: This work demonstrates the value of objective, executable evaluation methodologies with binary, reproducible scoring. The four-category response taxonomy (correct, format violation, hallucination, refused) provides a more nuanced understanding of LLM failure modes than simple correct/incorrect classifications. Compared to the abstract's summary of raw numbers, the full analysis additionally reveals that (a) performance degradation is not uniform but concentrated in specific specialised domains, (b) format violations are a distinct and relatively minor failure mode, and (c) the gap between models widens dramatically as domain specialisation increases. All code and data are publicly available.[1]

[1] https://github.com/ArunGMR0411/ai-hallucination-eval

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, et al. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[3] Aidan Gomez. Introducing Command R7B: Fast and efficient generative AI. Cohere Blog, 2024. https://cohere.com/blog/command-r7b. Accessed: 2025-11-01.

[4] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

[5] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[6] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 3214–3252, 2022.

[7] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[8] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1906–1919, 2020.

[9] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, et al. FACTScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 12076–12100, 2023.

[10] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.